

OWASP Juice Shop

Web Application Security Assessment

Assessment Type: Web Application Security Assessment

Target: OWASP Juice Shop

Target URL: http://localhost:3000

Testing Environment: Local / Controlled Lab

Primary Tool: OWASP ZAP 2.17.0

Assessment Status: Completed

1. Executive Summary

This assessment evaluated the security posture of the intentionally vulnerable OWASP Juice Shop application deployed in a controlled local environment. The assessment was performed using OWASP ZAP (Zed Attack Proxy) with the target restricted to `http://localhost:3000`.

The assessment included application reconnaissance, site discovery, spidering, client-side crawling, passive scanning, active scanning, and manual validation of selected security findings.

OWASP ZAP identified multiple alert instances during the assessment. These alerts were reviewed and grouped by finding type rather than being treated as individual vulnerabilities. Findings associated with external third-party resources were excluded from the assessment scope.

A SQL Injection finding affecting the product search functionality was manually validated. Supplying a malformed input to the `q` parameter resulted in a database-related SQLite error, demonstrating that user-controlled input reaches the application's database query processing without adequate input handling.

Additional security findings identified during the assessment included missing Content Security Policy protections, a CSP configuration issue, a potential missing anti-clickjacking protection, session identifiers appearing in URL query parameters, application error information disclosure, private IP address disclosure, and timestamp disclosure.

The assessment was conducted entirely against the local Juice Shop instance. No testing was intentionally performed against external systems or third-party infrastructure.

The detailed findings, evidence, impact considerations, and remediation recommendations are documented in the following sections of this report.

2. Assessment Overview

2.1 Objective

The objective of this assessment was to perform a structured web application security assessment of the OWASP Juice Shop application using OWASP ZAP.

The assessment focused on:

- Discovering application endpoints and functionality.
- Identifying potential security weaknesses.
- Performing automated passive and active security testing.
- Manually validating selected findings.
- Documenting evidence for identified issues.
- Providing practical remediation recommendations.

2.2 Target Application

The application assessed was OWASP Juice Shop, an intentionally vulnerable web application designed for security testing and training.

The application was deployed locally and accessed through:

`http://localhost:3000`

2.3 Scope

The assessment scope was strictly limited to:

`http://localhost:3000`

Testing activities included the local Juice Shop web application and its locally accessible API and application endpoints.

No intentional security testing was performed against systems outside the local Juice Shop environment.

2.4 Out-of-Scope Resources

During automated scanning, OWASP ZAP identified requests to external resources, including Azure Edge infrastructure.

These external resources were not part of the assessment scope and were therefore excluded from the vulnerability assessment conclusions.

Examples of excluded alert types included findings associated with:

- Cross-Domain Misconfiguration
- Strict-Transport-Security Header Not Set
- X-Content-Type-Options Header Missing
- Re-examine Cache-control Directives

These alerts were not treated as vulnerabilities in the local Juice Shop assessment.

2.5 Assessment Environment

Component	Details
Application	OWASP Juice Shop
Target	<code>http://localhost:3000</code>
Operating System	Windows 11

Security Tool	OWASP ZAP 2.17.0
Browser	Microsoft Edge
Deployment	Docker
Container Port	3000
ZAP Proxy	localhost : 8080
Testing Type	Local controlled assessment

3. Tools and Technologies

The following tools and technologies were used during the assessment:

Tool / Technology	Purpose
OWASP Juice Shop	Intentionally vulnerable target application
OWASP ZAP 2.17.0	Web application security testing and vulnerability scanning
Docker	Local deployment of OWASP Juice Shop
Microsoft Edge	Browser-based application interaction
Windows 11	Assessment workstation
ZAP Spider	Automated application crawling and endpoint discovery
ZAP Client Spider	Client-side application discovery
ZAP Passive Scanner	Detection of security issues through observed HTTP traffic

ZAP Active Scanner	Automated security testing using active requests
--------------------	--

Manual HTTP inspection	Validation of selected ZAP findings
------------------------	-------------------------------------

4. Assessment Methodology

The assessment followed a structured web application security testing workflow.

4.1 Reconnaissance

The initial phase involved launching the locally deployed OWASP Juice Shop application and confirming that it was accessible at:

`http://localhost:3000`

OWASP ZAP was then configured as the interception and assessment proxy.

The application's site structure and discovered resources were reviewed through the ZAP Sites tree.

4.2 Spider / Crawling

The ZAP Spider was used to automatically discover application resources and URLs.

The crawling phase helped identify accessible application functionality and endpoints that could subsequently be examined during passive and active scanning.

A Client Spider was also used to discover resources and functionality that rely on client-side application behavior.

4.3 Passive Scanning

Passive scanning was performed while application traffic was observed through OWASP ZAP.

Passive scanning analyzes HTTP requests and responses without intentionally modifying application behavior through attack payloads.

The passive scan identified several potential security issues, including security-header-related findings and information disclosure issues.

4.4 Active Scanning

An OWASP ZAP Active Scan was performed against the local target:

`http://localhost:3000`

The scan used the default scanning policy with recursive scanning enabled.

The completed active scan generated:

- **14,379 requests**
- **222 alert instances**
- **15 alert types**

The alert count represents individual alert instances generated during scanning. It does not represent 222 separate vulnerabilities.

4.5 Manual Validation

Selected automated findings were manually reviewed using the HTTP request and response information available within OWASP ZAP.

Manual validation was used to distinguish findings that could be directly demonstrated from findings that required additional contextual assessment.

The SQL Injection finding affecting the product search functionality was manually validated.

A request using the search parameter produced a database-related SQLite error when a malformed quote character was supplied, providing direct evidence that the supplied input reached database query processing.

The DOM-based XSS behavior was manually demonstrated using a controlled JavaScript payload supplied through the Juice Shop search URL fragment.

The CSRF behavior was manually demonstrated using a locally hosted attacker page on <http://localhost:8000>. The page submitted a state-changing POST request to <http://localhost:3000/profile> while the test browser was authenticated, and the username changed successfully to `CSRF_Attack_Demo`. The captured request contained `Origin: http://localhost:8000` and `Referer: http://localhost:8000/` and did not contain a visible CSRF token.

Other findings were assessed using their associated request, response, risk level, confidence level, and available evidence.

4.6 Finding Classification

The assessment findings were classified into the following categories:

Confirmed Finding

A finding for which manual validation provided sufficient evidence to demonstrate the reported security behavior.

Automated Finding Requiring Contextual Review

A finding reported by OWASP ZAP where the observed behavior indicates a potential security weakness, but the practical impact depends on the specific application context.

Informational Finding

A ZAP alert that provides useful security information but does not by itself represent a confirmed vulnerability.

Out-of-Scope Finding

An alert associated with an external resource that was not part of the defined assessment target.

Out-of-scope findings were excluded from the assessment conclusions.

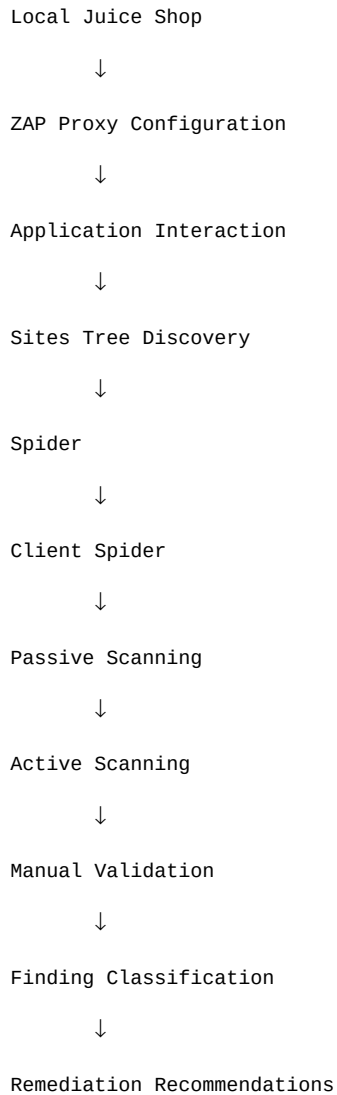
5. Reconnaissance Results

The reconnaissance phase established the local OWASP Juice Shop application as the assessment target and identified its accessible application structure.

The ZAP Sites tree provided an overview of discovered resources and endpoints.

The standard Spider and Client Spider were used to expand application coverage before security scanning.

The reconnaissance process established the following assessment flow:



6. Vulnerability Assessment Results

The assessment identified several security-related findings within the local OWASP Juice Shop application.

The findings below are presented by security relevance and validation status. Repeated alert instances generated by

6.1 SQL Injection

****Risk:**** High

****Confidence:**** Low according to ZAP; manually validated during assessment

****CWE:**** CWE-89 – Improper Neutralization of Special Elements used in an SQL Command

****Affected Endpoint:****

``http://localhost:3000/rest/products/search``

****Affected Parameter:****

``q``

****Finding Type:**** SQL Injection

Description

OWASP ZAP identified a potential SQL Injection vulnerability in the product search functionality.

The affected endpoint accepts user-controlled input through the ``q`` query parameter.

The finding was manually validated by sending controlled input to the search endpoint.

A normal search request such as:

``q=apple``

returned a successful response.

When a single quote was appended to the input, the application returned an HTTP 500 error containing a SQLite database error.

``SQLITE_ERROR: near "'%': syntax error``

The response also disclosed that the application is using SQLite for database processing.

Validation

The following request was used during validation:

GET /rest/products/search?q=apple HTTP/1.1

Host: localhost:3000

7. Informational Findings

Several OWASP ZAP alerts were classified as informational rather than confirmed vulnerabilities.

7.1 Modern Web Application

OWASP ZAP identified the application as a modern web application.

This is an informational classification that reflects characteristics of the application's client-side architecture.

It does not, by itself, represent a security vulnerability.

The related evidence is documented in:

06-Evidence/screenshots/14-modern-web-application.png

7.2 Session Management Response Identified

OWASP ZAP identified a response associated with session management functionality.

This alert is informational and does not, by itself, demonstrate a vulnerability.

The finding should be interpreted together with the application's actual session-management implementation.

7.3 User Agent Fuzzer

OWASP ZAP reported a User Agent Fuzzer systemic alert.

This is an informational/systemic scan result rather than a confirmed vulnerability.

It indicates that ZAP performed testing involving the HTTP User-Agent header.

8. Out-of-Scope / Excluded Findings

The assessment was strictly limited to the local Juice Shop instance:

`http://localhost:3000`

During automated scanning, some requests were made to external resources used by the application.

These resources were not part of the authorized assessment scope.

Accordingly, findings associated with the following external resources were excluded from the final vulnerability conclusions:

- Azure Edge / CDN resources

- External static configuration resources
- Other third-party resources outside localhost : 3000

Examples of excluded ZAP alert types included:

Alert	Reason for Exclusion
Cross-Domain Misconfiguration	Associated with an external Azure Edge resource
Strict-Transport-Security Header Not Set	Associated with an external HTTPS resource
X-Content-Type-Options Header Missing	Associated with an external Azure Edge resource
Re-examine Cache-control Directives	Associated with an external Azure Edge resource

These exclusions are important because automated scanners can follow external resources encountered during application interaction. An alert generated for such a resource should not automatically be attributed to the assessed application.

No remediation recommendation is made for these excluded third-party resources.

9. Risk Summary

The following table summarizes the principal findings identified during the assessment.

Finding	Risk	Validation / Classification
SQL Injection	High	Manually validated
Content Security Policy Header Not Set	Medium	Directly verified
CSP Directive Configuration Issue	Medium	Automated finding requiring review
Missing Anti-clickjacking Header	Medium	Automated finding requiring contextual review

Session ID in URL Rewrite	Medium	Automated finding with supporting request evidence
Application Error Disclosure	Low	Manually reviewed
Private IP Disclosure	Low	Direct response evidence
Timestamp Disclosure	Low	Low-confidence automated finding

Risk Interpretation

High

A high-risk finding may have significant security consequences if exploitable in the application's deployment context.

The SQL Injection finding was the highest-priority technical finding identified during this assessment because manually supplied input produced a database-level error.

Medium

Medium-risk findings represent security weaknesses that may provide meaningful additional attack surface or reduce defense-in-depth protections.

The CSP, anti-clickjacking, and session-management findings fall into this category based on OWASP ZAP's reported risk classifications.

Low

Low-risk findings generally provide information or represent weaknesses with more limited direct security impact.

The application error disclosure, private IP disclosure, and timestamp disclosure findings fall into this category.

Important Qualification

Risk classifications reported by automated security tools should be interpreted together with application context.

A scanner alert does not automatically demonstrate successful exploitation.

For this assessment, manual validation was used where practical to distinguish directly demonstrated behavior from automated findings requiring additional contextual review.

10. Remediation Recommendations

The following recommendations are based on the findings identified during the assessment.

Remediation should be prioritized according to the security impact, exploitability, and relevance of each finding to the application's intended deployment environment.

10.1 Priority 1 — Address SQL Injection

Finding: SQL Injection

Risk: High

CWE: CWE-89

The product search functionality should be reviewed and corrected to prevent user-controlled input from being interpreted as part of a database query.

Recommended Actions

1. Replace dynamically constructed SQL statements with parameterized queries or prepared statements.
2. Ensure user input is never concatenated directly into SQL statements.
3. Apply appropriate input validation where applicable.
4. Use a database account with only the privileges required by the application.
5. Implement centralized database error handling.

6. Prevent SQL/database error details from being returned to users.
7. Perform regression testing after remediation.

Verification

After remediation, repeat controlled security testing against the affected search functionality and confirm that malformed input no longer produces database syntax errors or changes query behavior.

10.2 Priority 2 — Implement a Content Security Policy

Finding: Content Security Policy Header Not Set

Risk: Medium

The application should implement an appropriately restrictive Content Security Policy.

Recommended Actions

1. Define a CSP policy appropriate for the application's legitimate resources.
2. Explicitly define required resource directives.
3. Avoid unnecessarily broad source expressions.
4. Test the policy against all application functionality.
5. Consider deploying the policy initially in reporting/monitoring mode during development.
6. Gradually strengthen the policy after legitimate resource requirements are understood.

Verification

Inspect application responses and confirm that the expected Content-Security-Policy header is present and correctly configured.

10.3 Priority 3 — Strengthen Clickjacking Protection

Finding: Missing Anti-clickjacking Header

Risk: Medium

The application should explicitly define its framing policy for browser-rendered pages.

Recommended Actions

1. Review which application pages are intended to be frameable.
2. Use CSP `frame-ancestors` to define permitted framing origins.
3. Consider `X-Frame-Options` where appropriate for compatibility.
4. Apply the protection to relevant HTML responses.
5. Review the Socket.IO-related alert separately because the reported response is not a conventional HTML page.

Verification

Confirm that sensitive browser-rendered pages cannot be embedded by unauthorized origins.

10.5 Priority 5 — Review Session Identifier Handling

Finding: Session ID in URL Rewrite

Risk: Medium

The application's session-management implementation should be reviewed to determine whether the `sid` value exposed in Socket.IO URLs represents security-sensitive session state.

Recommended Actions

1. Determine the purpose and sensitivity of the `sid` parameter.
2. Avoid exposing authentication credentials in URLs.

3. Use secure session-management mechanisms where applicable.
4. Apply appropriate expiration and rotation controls.
5. Prevent sensitive session information from unnecessary logging.
6. Review browser history, proxy logging, and monitoring implications.

Verification

Confirm that session identifiers do not provide unauthorized authentication or authorization capabilities if exposed through URLs.

10.6 Priority 6 — Reduce Information Disclosure

Findings:

- Application Error Disclosure
- Private IP Disclosure
- Timestamp Disclosure

Risk: Low

The application should minimize information exposed through client-accessible responses.

Recommended Actions

1. Disable detailed stack traces in production.
2. Return generic error messages to clients.
3. Log detailed errors securely on the server.
4. Remove unnecessary internal IP addresses from client-facing configuration.
5. Review static-resource metadata and timestamps.
6. Avoid exposing internal filesystem paths.

7. Review all configuration endpoints for unnecessary information exposure.

Verification

Repeat requests against the affected endpoints and confirm that internal implementation details, private infrastructure information, and unnecessary debugging information are no longer exposed.

10.7 General Security Improvements

In addition to the specific findings above, the following general practices should be considered:

- Keep application dependencies updated.
- Use secure configuration management.
- Separate development and production configurations.
- Apply least-privilege principles to application and database accounts.
- Implement centralized logging and monitoring.
- Perform regular dependency and vulnerability scanning.
- Review HTTP security headers as part of application hardening.
- Conduct periodic security testing after significant application changes.

11. Evidence

Evidence collected during the assessment is stored within the project evidence directory.

The primary screenshot evidence is located at:

06-Evidence/screenshots/

The evidence currently includes screenshots covering:

- Juice Shop availability

- ZAP Sites Tree
- Spider results
- Client Spider results
- Passive scan results
- CSP findings
- Missing anti-clickjacking finding
- Session ID in URL rewrite
- Application error disclosure
- Private IP disclosure
- Timestamp disclosure
- Modern web application classification

A dedicated SQL Injection screenshot can be added during the final evidence review if required.

The assessment also relied on HTTP request and response information observed directly within OWASP ZAP during manual validation.

12. Limitations

The assessment was performed against an intentionally vulnerable application deployed in a controlled local environment.

The following limitations apply:

1. The assessment was restricted to `http://localhost:3000`.
2. External third-party resources identified during scanning were excluded.
3. Automated scanner results were not automatically treated as confirmed vulnerabilities.
4. Manual validation was performed only for selected findings.
5. No destructive testing was performed.

6. No attempt was made to extract or modify unauthorized application data.
7. The assessment represents the state of the application during the testing period and does not guarantee the absence of other vulnerabilities.
8. Findings associated with external infrastructure were not assessed because they were outside the defined scope.

13. Conclusion

The OWASP Juice Shop application was successfully assessed using OWASP ZAP within a controlled local testing environment.

The assessment included reconnaissance, application crawling, passive scanning, active scanning, and manual validation.

The testing produced 14,379 requests and 222 alert instances across 15 alert types. These alerts were reviewed and consolidated by finding type.

The principal manually validated findings included SQL Injection in the product search functionality, DOM-based XSS in the search functionality, and CSRF affecting the profile username-change functionality. Controlled SQL injection input resulted in a database-level SQLite error. The XSS test executed a controlled JavaScript payload in the browser. The CSRF test used a locally hosted attacker page to submit a state-changing cross-origin request that changed the authenticated username.

Additional security weaknesses identified during the assessment included missing or incomplete security headers, session identifier exposure in URLs, application error disclosure, private IP disclosure, and timestamp disclosure.

The assessment also demonstrated the importance of reviewing automated scanner results in application context. Several alerts were informational, while others were associated with external resources and were therefore excluded from the assessment scope.

The remediation recommendations provided in this report focus on reducing the application's attack surface, improving input handling, strengthening browser security controls, protecting session information, and minimizing unnecessary information disclosure.

14. Appendix

Appendix A — Target URLs

The primary assessment target was:

`http://localhost:3000`